USF: Unit-Streamed Fine-Tuning of Large Language Models with Exact Gradients on Consumer GPUs

***Abstract*—Fine-tuning a 7B-parameter language model conventionally requires 8–16 GB of GPU memory, because the frozen base must remain resident throughout the forward and backward passes. We show that this requirement is an artifact of implementation rather than a mathematical necessity. Reverse-mode differentiation composes *local* vector–Jacobian products: the gradient at layer i depends only on that layer's input, its weights, and the incoming cotangent—never on the other $L-1$ layers. We introduce *Unit-Streamed Fine-Tuning* (USF), which keeps exactly one weight unit resident at a time, streamed from disk, and prove that the resulting gradients are *exact* (Theorem 3) rather than approximate. The consequence is a memory bound independent of model depth and of total parameter count: only the largest single unit matters (Theorem 7). We further prove that inverting the loop order (*layer-major*, Theorem 13) divides disk traffic by the gradient-accumulation factor G while leaving gradients bit-identical, shifting the regime from I/O-bound to compute-bound. On a GTX 1050 (3.94 GB) we fine-tune Qwen2.5-7B—15.2 GB of weights on disk—at an allocator peak of 2.2 GB, or 2.9 GB of total device occupancy including the CUDA context, reducing held-out perplexity from 9.52 to 4.46; layer-major is verified gradient-exact on the 7B model itself ($\max|\Delta| = 0$ over 1400 tensors) and yields a measured $1.40\times$ speed-up. A predictive cost model matches measurements within 6%. USF is adapter-agnostic: a spectral adapter and LoRA both run unmodified. Under the same budget, 4-bit quantization does not fit (5.43 GB required), making this a feasibility separation rather than a preference. The bound predicts that Llama-3-70B fits the same card at the implemented granularity; we state precisely what this does and does not establish.**

***Index Terms*—Parameter-efficient fine-tuning, memory-constrained training, large language models, exact gradients, out-of-core computation, automatic differentiation, consumer hardware.**

I. Introduction

PARAMETER-EFFICIENT fine-tuning (PEFT) reduced the *trainable* footprint of adapting large language models by orders of magnitude: LoRA [1] updates well under 1% of the weights, and adapters [2] or prompt tuning [3] update fewer still. The *resident* footprint, however, did not follow. Whatever the adapter, the frozen base must be present during both the forward and the backward pass, so adapting a 7B model in half precision still demands 8–16 GB of device memory. The practical effect is a hardware census: researchers whose GPU holds 4 GB are excluded from working with 7B models at all, irrespective of how few parameters they intend to train.

Quantization attacks the constant [4], [5], compressing the resident base by a factor of four to eight. It does not attack the coupling. A 4-bit 7B model still occupies several gigabytes, a 4-bit 70B model still exceeds any consumer card, and the compression is paid for in accuracy. Offloading frameworks such as ZeRO-Infinity [6] do move parameters off the device, but they were designed for a different regime—training *all* weights across a cluster—where the dominant problem is sharding optimizer state.

Manuscript prepared July 2026. All experiments were carried out on a single NVIDIA GTX 1050 (3.94 GB) desktop machine.

A. The observation

This paper begins from an elementary property of reverse-mode differentiation [7], [8] that, to our knowledge, has not been exploited to its logical conclusion in this setting. Backpropagation is a composition of *local* vector–Jacobian products (VJPs). To differentiate through layer i one requires that layer's input, that layer's weights, and the incoming cotangent—and nothing else. The remaining $L-1$ layers are irrelevant *at that instant*. Keeping the whole model resident is therefore a choice made for speed, not a requirement imposed by correctness.

We take that observation literally. USF holds exactly one weight *unit* in device memory at a time, streams it from disk, computes, releases it, and re-streams it during the backward pass. The immediate question is whether this costs accuracy. It does not: we prove (Theorem 3) and verify ($\max|\Delta| = 0$) that the gradients are identical to those of ordinary full-graph autograd. USF is not an approximation.

B. The consequence

Once residency is decoupled from correctness, the memory bound changes shape. The peak is set by the largest *single* unit, not by the model:

$$\mathrm{VRAM}_{\mathrm{peak}} \;\approx\; \max_{u} |W_u| \cdot s \;+\; |\theta| \cdot s_{\mathrm{opt}} \;+\; A, \qquad (1)$$

which depends neither on the depth L nor on the total size $\sum_i |W_i|$ (Theorem 7 and its corollaries). Adding layers to a model does not increase its memory requirement; it increases only time and disk traffic. Model size ceases to be a memory problem and becomes a storage-and-time problem. We regard this decoupling, rather than any individual engineering component, as the contribution of this work.

A second result follows from the same locality. Because gradient accumulation is a sum, and sums are order-invariant, the two nested loops of a training step—over micro-batches and over layers—may be exchanged. Keeping a unit resident while all G micro-batches pass through it divides disk traffic by G with bit-identical gradients (Theorem 13). This reordering is pointless when the model fits in memory, which is presumably

why the literature does not consider it; under streaming it is the dominant optimization, and it moves the system from I/O-bound to compute-bound.

C. Contributions

1) **A size-independence result.** We prove that the memory required to fine-tune a frozen model is a function of its largest streamable unit alone, independent of depth and of total parameter count (Theorem 7, Corollary 8, Corollary 9).
2) **Exactness, proved and verified.** USF preserves gradients exactly (Theorem 3); we measure $\max|\Delta| = 0$, including on the 7B model itself (Section XI).
3) **Layer-major streaming.** Exchanging the loop order divides I/O by the accumulation factor G at no cost in correctness (Theorem 13), changing the performance regime (Section IX).
4) **A predictive cost model** that matches measurement within 6% (Section IX), which is what licenses our scaling statements as predictions rather than extrapolations.
5) **An adapter- and model-agnostic system.** The same infrastructure runs LoRA and a spectral adapter unmodified (Corollary 5), and selects its streaming granularity automatically from the available memory (Section X).

We are explicit about scope. We *demonstrate* 7B on a 4 GB card; we *predict*, from a proved bound and a validated cost model, that 70B fits the same card with the machinery we implement; and we identify precisely which unimplemented extension a 405B model would require (Section XII). We do not claim the latter two as results.

II. Related Work

Parameter-efficient fine-tuning. LoRA [1] and its descendants [9], adapters [2], and prompt tuning [3] minimise *trainable* parameters. They are orthogonal to this work: USF governs where computation lives, not what is adapted, and Corollary 5 makes this formal. We evaluate LoRA and a spectral adapter under USF without modifying either.

Quantization. QLoRA [4] and LLM.int8() [5] shrink the resident base by compressing it. The axis is different from ours in two respects. First, quantization buys memory with *accuracy*, whereas USF buys it with *time* and is exact (Corollary 4). Second, it improves the constant but preserves the coupling to model size; Section XI-F reports the concrete consequence, namely that 4-bit is infeasible on our budget while USF is not. The two are composable, and we do not present them as alternatives.

Offloading and sharding. ZeRO [10], ZeRO-Offload [11] and ZeRO-Infinity [6] established that parameters and optimizer state can reside off-device, and are the closest prior art. The regime differs on the point that matters. Those systems target training in which every weight is trainable, distributed over a cluster; their central difficulty, and their main contribution, is partitioning optimizer state, whose size is proportional to the trainable parameter count. In our regime $|\theta| \ll |W|$, so there is no optimizer state worth partitioning, and the problem collapses entirely onto the residency of W. What we add is therefore not offloading per se, but (i) the formal bound and its independence of model size (Theorem 7), (ii) the granularity hierarchy that determines feasibility (Theorem 10), and (iii) the layer-major reordering (Theorem 13), which has no purpose in a regime where the model is resident. Production dispatch libraries such as `accelerate` [12] provide module-level CPU/disk placement aimed principally at inference; Section XI-F reports our measurements with that path.

Recomputation. Activation checkpointing [13] and the classical REVOLVE schedules [14] trade compute for memory by discarding and recomputing *activations*. This is orthogonal and complementary: we use checkpointing, and additionally discard and re-stream the *weights*. To our knowledge the storage/recomputation trade-off has not previously been applied to frozen weights with an accompanying exactness guarantee.

I/O-aware design. FlashAttention [15] demonstrated that reorganising a computation around its memory traffic—without altering its result—yields substantial gains, and that exactness is worth preserving rather than trading away. USF shares that philosophy at a different level of the hierarchy: FlashAttention minimises traffic between on-chip and high-bandwidth memory within attention, whereas USF governs traffic between disk and device for the weights of the entire model. Theorem 13 is an I/O-aware scheduling result in the same spirit, and Section IX follows the roofline reasoning of [16] in identifying which resource binds.

Pipelining. GPipe [17] and related approaches [18] partition layers across devices and overlap micro-batches to fill the pipeline. Superficially, layer-major (Theorem 13) resembles a pipeline schedule; the purpose is inverted. Pipelining distributes a model too large for one device across many; layer-major keeps one device and amortises the cost of *re-reading* the model, an expense that only exists when weights are streamed.

III. Preliminaries

Let a frozen transformer of L layers compute $h_0 = E(x)$, $h_i = f_i(h_{i-1}; W_i, \theta_i)$ for $i = 1, \dots, L$, and $\ell = C(h_L, y)$, where W_i are the *frozen* weights of layer i and θ_i the *trainable* adapter parameters. Because the base is frozen, $\nabla_{W_i}\ell$ is never required—only $\nabla_\theta \ell$. We assume the PEFT regime $|\theta| \ll |W|$ throughout, and we write s for bytes per scalar ($s = 2$ in fp16), B for micro-batch size, S for sequence length, d for model width, and G for the number of gradient-accumulation steps.

Definition 1 (Streamable unit)**.** *A* unit u *is a subset of weights that can be loaded and released independently, and* $\mathcal{U}$ *denotes a partition of* $\{W_i\}$ *into such units. We consider three nested granularities:* layer *(an entire decoder block),* sublayer *(the attention block and the MLP block separately), and* matrix *(a single projection).*

Definition 2 (Streaming operators)**.** LOAD(u) *materialises* W_u *from disk into device memory at an I/O cost of* $|W_u|$ *bytes;* FREE(u) *returns* W_u *to a placeholder device, releasing the memory at no I/O cost. The two are strictly nested: no other unit is loaded between a* LOAD(u) *and its matching* FREE(u).

Algorithm 1 USF (batch-major, layer granularity)

$h \leftarrow E(x)$
for $i = 1$ **to** L **do** ▷ forward, no graph
  $c_i \leftarrow h$ ▷ checkpoint: input of layer i
  LOAD(W_i); $h \leftarrow f_i(h; W_i, \theta_i)$; FREE$(W_i)$
end for
$\ell \leftarrow C(h, y)$; $g \leftarrow \partial\ell/\partial h_L$
for $i = L$ **down to** 1 **do** ▷ backward, re-streaming
  $x \leftarrow c_i$ with gradient tracking enabled
  LOAD(W_i)
  $\hat{h} \leftarrow f_i(x; W_i, \theta_i)$ ▷ recompute *with* graph
  $\hat{h}$.BACKWARD(g) ▷ accumulate $\partial\ell/\partial\theta_i$
  FREE(W_i); $g \leftarrow x.\mathrm{grad}$
end for

Algorithm 1 states the base algorithm. The forward pass builds no autograd graph: activations *and* weights are discarded, and each layer is recomputed—with its weights re-streamed— during the backward pass.

IV. EXACTNESS

The defining property of USF is that streaming is not an approximation. This separates it categorically from quantization, which trades memory for accuracy.

Theorem 3 (Gradient exactness)**.** *Let $\nabla_\theta \ell^{\text{full}}$ be the gradient produced by reverse-mode differentiation over the complete computational graph with every W_i resident, and let $\nabla_\theta \ell^{USF}$ be the gradient produced by Algorithm 1. Then, in exact arithmetic,*

$$\nabla_\theta \ell^{USF} \;=\; \nabla_\theta \ell^{\text{full}}. \tag{2}$$

Proof: Reverse-mode differentiation computes, for each layer i, the pair

$$\left(\frac{\partial \ell}{\partial \theta_i},\ \frac{\partial \ell}{\partial h_{i-1}}\right) = \mathrm{VJP}_{f_i}\left(h_{i-1}, W_i, \theta_i;\ \frac{\partial \ell}{\partial h_i}\right). \tag{3}$$

The map in (3) is *local*: it is a function of (a) the evaluation point h_{i-1}, (b) the parameters W_i, θ_i, and (c) the incoming cotangent $\partial\ell/\partial h_i$ alone. It does not depend on W_j for any $j \neq i$.

Algorithm 1 supplies all three unchanged. For (a), the checkpoint $c_i = h_{i-1}$ is recorded during the forward pass, so the recomputation is evaluated at the identical point, with no approximation. For (b), LOAD(W_i) restores W_i bit-for-bit, since it re-reads the same tensor from disk, while θ_i is resident and unmodified within a step. For (c), $g = \partial\ell/\partial h_i$ holds by backward induction from $g_L = \partial\ell/\partial h_L$.

Every local VJP is therefore evaluated at exactly the point at which it would be evaluated in the full graph, and their composition—which is precisely the chain rule—yields the same value. Finally, FREE(W_i) alters only where a tensor resides, not its value; arithmetic is invariant to device placement. ∎

Corollary 4 (Not an approximation)**.** *USF introduces no error, bias, or variance. Any deviation from the full-graph gradient is confined to the ordinary non-determinism of floating-point arithmetic (Remark 6).*

Corollary 5 (Adapter agnosticism)**.** *Theorem 3 uses no property of f_i beyond differentiability. USF is therefore independent of the adapter: it applies verbatim to LoRA, to spectral adapters, or to any parameterisation of θ. We verify this with two different adapters in Section XI.*

Remark 6 (Floating-point considerations)**.** *Theorem 3 asserts equality in exact arithmetic; two points deserve care in practice. First, weights are preserved exactly:* LOAD *re-reads the same fp16 tensor, so no requantisation occurs, and this holds unconditionally. Second, if the recomputation selects a different kernel from the original forward, floating-point reduction order may differ in the last bits. This is the non-determinism inherent to standard activation checkpointing [13] and is not specific to USF. In our measurements, with identical shapes and kernels, agreement is exact (Table III).*

V. THE MEMORY BOUND

Theorem 7 (Peak memory)**.** *Under Algorithm 1 with granularity $\mathcal{U}$, the peak device memory satisfies*

$$\mathrm{VRAM}_{peak} \;\le\; \underbrace{\max_u |W_u| \cdot s}_{\text{resident unit}} + \underbrace{|\theta| \cdot s_{opt}}_{\text{adapters}} + \underbrace{A}_{\text{activations}} + \underbrace{K}_{\text{checkpoints}}, \tag{4}$$

where A is the internal activation memory of a single *unit and $K = L \cdot B \cdot S \cdot d \cdot s$ if the checkpoints $\{c_i\}$ reside on device.*

Proof: At every instant of Algorithm 1 exactly one unit is loaded, since LOAD and FREE are strictly nested and non-overlapping (Definition 2); this yields the first term. The adapters are resident by construction: θ is updated every step and its optimizer state must persist across steps; its size is $|\theta| \ll |W|$ by the PEFT hypothesis. A gradient-carrying graph exists only within the iteration of one unit, because the forward pass builds no graph; this yields A. Finally, the c_i are the only tensors that survive between iterations. ∎

Corollary 8 (Independence of depth)**.** *The checkpoints c_i are activations, not weights, and are not required on device between their production and their use. Offloading them to host memory gives $K \approx 0$ and hence*

$$\mathrm{VRAM}_{peak} \;\approx\; \max_u |W_u| \cdot s + |\theta| \cdot s_{opt} + A, \tag{5}$$

which is independent of L. Adding layers to a model does not increase its memory requirement.

Corollary 9 (Independence of model size)**.** *The bound does not contain $\sum_i |W_i|$. Two models whose largest unit has the same size have the same memory requirement, however different their total parameter counts.*

Corollary 9 is the paper's central claim, and it is worth stating what it does and does not say. It does not say that a larger model is as cheap to fine-tune as a smaller one: the I/O and compute costs both scale with $\sum_i |W_i|$, as Section IX quantifies. It says that those costs are paid in *time*, not in *memory*. The device requirement is set by the largest

TABLE I
UNIT SIZE BY GRANULARITY (FP16). THE ADAPTERS, ACTIVATIONS AND CHECKPOINTS ADD UNDER 0.6 GB. VALUES ARE COMPUTED FROM THE PUBLISHED ARCHITECTURE CONFIGURATIONS BY THE SAME CODE THAT DRIVES THE STREAMER.

Model	L	**layer**	**attn/mlp**	**matrix**	**4 GB?**
Qwen2.5-7B	28	0.43 GB	0.05 / 0.38	0.13 GB	any granularity
Llama-3-8B	32	0.41 GB	0.08 / 0.33	0.11 GB	any granularity
13B-class	40	0.59 GB	0.20 / 0.40	0.13 GB	any granularity
Llama-3-70B	80	1.59 GB	0.28 / 1.31	0.44 GB	**layer**
Llama-3-405B	126	5.94 GB	1.06 / 4.88	**1.62 GB**	matrix only†

† Matrix granularity is not implemented; see Remark 11.

single unit and by nothing else, which is why the granularity hierarchy of the next section determines feasibility.

VI. GRANULARITY

Theorem 10 (Granularity hierarchy)**.** *For the nested partitions of Definition 1,*

$$\max_{i,p} |W_{i,p}| \;\le\; \max_{i} \max\Big(|W_i^{attn}|, |W_i^{mlp}|\Big) \;\le\; \max_{i} |W_i|, \tag{6}$$

and Theorem 7 applies to any of them. Refining the granularity reduces peak memory monotonically at identical total I/O.

Proof: The partitions are nested (matrix $\subset$ sublayer $\subset$ layer), and a maximum over a finer partition cannot exceed the maximum over a coarser one. The total I/O per pass is $\sum_u |W_u| = \sum_i |W_i|$, which is invariant to the partition: the same bytes are read, in smaller pieces. ∎

Table I instantiates Theorem 10 for four model families. Two observations follow. Llama-3-70B [19] fits a 3.94 GB card at *layer* granularity, with 1.59 GB per block; no refinement is needed. A 405B model does not fit at layer or sublayer granularity, but its largest single matrix occupies 1.62 GB and would fit—which is the content of Corollary 9 at its extreme.

Remark 11 (An implementation limit, stated plainly)**.** *Theorem 10 is a statement about partitions and holds at matrix granularity. Our* implementation*, however, supports layer and sublayer granularity only. The reason is engineering rather than mathematics: we delegate attention to the reference implementation [20], which consumes W_q, W_k, W_v, W_o jointly within a single call. Streaming them individually would require reimplementing the attention internals, including rotary embeddings [21] and grouped-query attention [22]—a substitution we judged to carry more risk of silent incorrectness than the benefit warrants. We therefore claim 70B by the bound at an implemented granularity, and we claim nothing for 405B beyond Table I.*

VII. LAYER-MAJOR STREAMING

Algorithm 1 is *batch-major*: for each micro-batch it traverses all L units. With accumulation over G micro-batches, the model is therefore read G times per optimizer step. The following reordering removes that factor.

Definition 12 (Layer-major)**.** *Exchange the two loops: hold unit u resident and pass all G micro-batches through it before advancing to $u+1$. The backward pass is symmetric—one load of W_i, then G local backpropagations.*

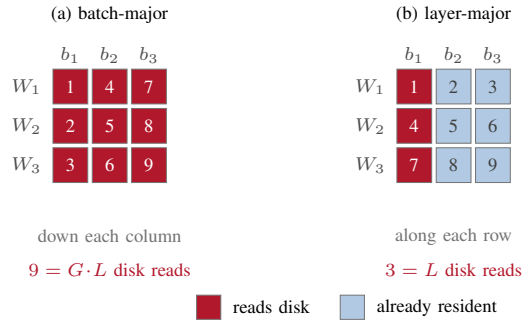

Fig. 1. Why inverting the loop order divides I/O by G (Theorem 13), drawn for L=3 units and G=3 micro-batches. Cells are (unit, micro-batch) visits, numbered in traversal order; both schedules perform all nine, so both compute the same gradient. Batch-major (a) finishes a micro-batch before moving on, so every visit re-reads its unit from disk. Layer-major (b) finishes a unit before moving on, so only the first visit in each row reads: the other $G-1$ find the weights already there. The count falls from $G{\cdot}L$ to L, and the price is G sets of boundary activations, which Corollary 8 places in host memory.

Fig. 1 shows why this matters. The two schedules visit exactly the same set of (unit, micro-batch) pairs, and by Theorem 3 each visit computes the same local VJP; they differ only in the order of the traversal, and therefore in how many of those visits find the unit already resident.

Theorem 13 (Exactness and I/O of layer-major)**.** *The layer-major schedule of Definition 12 produces gradients identical to Algorithm 1, and its I/O per optimizer step is*

$$\mathrm{I/O}_{\textit{layer-major}} = 2\sum_i |W_i| \quad \textit{versus} \quad \mathrm{I/O}_{\textit{batch-major}} = G{\cdot}2\sum_i |W_i|, \tag{7}$$

a reduction by the factor G.

Proof: Exactness. Gradient accumulation computes $\nabla_\theta \ell = \sum_{j=1}^{G} \nabla_\theta \ell^{(j)}$. Addition is commutative and associative, so the order in which the terms are accumulated does not affect the sum. Each $\nabla_\theta \ell^{(j)}$ is computed by the local VJP of Theorem 3, evaluated at $(c_i^{(j)}, W_i, \theta_i)$—the same points as in the batch-major schedule. Exchanging the loops changes the order of visits, not any evaluation point.

I/O. In Definition 12 the outer loop traverses i once per optimizer step, so each W_i is loaded exactly once per pass and twice in total (forward and backward), giving $2\sum_i |W_i|$. In Algorithm 1 the outer loop over j repeats the full traversal G times. ∎

The cost is that G sets of activations must be held at each unit boundary, $G \cdot B \cdot S \cdot d \cdot s$ bytes; for $G = 8$, $B = 4$, $S = 256$, $d = 3584$ in fp16 this is 59 MB per boundary, and by Corollary 8 it may be placed in host memory. The trade is therefore G-fold less I/O in exchange for G-fold more offloadable activations.

Proposition 14 (Choice of G)**.** *Under layer-major, the time per example is*

$$T_{ex}(G) = \frac{2\sum_i |W_i|}{\mathrm{BW} \cdot G \cdot B} + \frac{C}{B}, \tag{8}$$

with C the compute time of one micro-batch and BW *the disk bandwidth. T_{ex} is monotonically decreasing in G and converges to the compute-bound limit C/B. The largest G satisfying the activation-memory constraint is therefore optimal.*

Remark 15 (Why this is absent from prior work)**.** *Layer-major has no meaning when the model is resident: if W_i never leaves the device, loading it "once" rather than "G times" is not a distinction. The reordering becomes the dominant optimization precisely and only in the streaming regime, which is the regime this paper constructs.*

VIII. Prefetching

Proposition 16 (Overlap condition)**.** *Let $t_{io}(u) = |W_u|/\mathrm{BW}$ and let $t_{cmp}(u)$ be the compute time of unit u. If* LOAD$(u{+}1)$ *is issued asynchronously when the computation of u begins, then*

$$T_{step} \approx \max(\textstyle\sum_u t_{io}(u),\ \sum_u t_{cmp}(u)) + t_{io}(u_1), \qquad (9)$$

and the I/O is fully hidden whenever $t_{io}(u{+}1) \le t_{cmp}(u)$ for all u.

Proof: The schedule is a two-stage pipeline (load; compute) with independent buffers. In steady state its throughput is that of the slower stage, and the initial fill contributes $t_{\mathrm{io}}(u_1)$. ∎

Double buffering keeps two units resident, so the first term of (4) becomes $2\max_u|W_u|\cdot s$; for 7B at layer granularity this is 0.86 GB rather than 0.43 GB, which remains comfortable on a 3.94 GB card. With our calibrated constants (Section IX), one full traversal costs $\sum_u t_{\mathrm{io}} \approx 48\,\mathrm{s}$ against $\sum_u t_{\mathrm{cmp}} \approx 57\,\mathrm{s}$ for a single micro-batch at $B{=}4$, $S{=}256$, so the overlap condition holds—though only just, which is consistent with (12) placing the compute-bound threshold at 8.6×10^2 tokens and this configuration supplying 1024. Combined with Theorem 13, which divides t_{io} by G, the margin becomes ample. Prefetching alters when a tensor is read, never its value, so Theorem 3 is unaffected—a fact we verify rather than assume (Table III).

IX. Cost Model

Let $N = \sum_i |W_i|/s$ be the number of streamed parameters, BW the effective disk bandwidth, and Φ the effective device throughput. Using the standard estimate of $6N$ FLOPs per token for a combined forward and backward pass [23],

$$T_{\mathrm{io}} = 2Ns/\mathrm{BW} \quad \text{(layer-major)}, \qquad (10)$$

$$T_{\mathrm{cmp}} = 6N(G \cdot B \cdot S)/\Phi, \qquad (11)$$

with $T_{\mathrm{step}} = T_{\mathrm{io}} + T_{\mathrm{cmp}}$ without prefetching and $\max(T_{\mathrm{io}}, T_{\mathrm{cmp}})$ with it (Proposition 16). The system is compute-bound when $T_{\mathrm{cmp}} > T_{\mathrm{io}}$; substituting (10)–(11), the streamed size N cancels and the condition reduces to a statement about tokens alone,

$$G \cdot B \cdot S \;>\; \frac{s\,\Phi}{3\,\mathrm{BW}}. \qquad (12)$$

That N cancels is the substantive part: which resource binds is a property of the *schedule*, not of the model's size, so a larger model does not become more I/O-bound—it becomes uniformly slower. With our calibrated constants ($s = 2$, $\Phi = 0.7\,\mathrm{TFLOP/s}$, $\mathrm{BW} = 0.54\,\mathrm{GB/s}$) the threshold is $\approx 8.6 \times 10^2$ tokens per step. A layer-major step with $G = 8$, $B = 4$, $S = 256$ processes 8192 tokens and is therefore firmly compute-bound, by an order of magnitude—the reordering of Theorem 13 does not merely accelerate the system, it changes which resource limits it. This is the roofline reasoning of [16] applied to an out-of-core training loop.

TABLE II
Cost model versus measurement (Qwen2.5-7B, GTX 1050, $B = 4$, $S = 256$).

Quantity	Model	Measured	Error
I/O per step (batch-major)	24.3 GB	24.9 GB	2 %
Time per example (batch-major)	25.6 s	26.9 s	5 %
Layer-major speed-up (G=4)	1.49×	1.40×	6 %

Validation. We calibrated BW and Φ from two independent measurements ($B = 1$ and $B = 4$) and used the model to predict the remainder. Table II reports the outcome: the largest discrepancy is 6%. We rely on this agreement in Section XII, where the model is used predictively.

X. Automatic Granularity Selection

Theorems 7 and 10 yield an operative criterion. Given a memory budget V, traverse the granularities from coarsest to finest and return the first whose bound fits.

Proposition 17 (Optimality of the coarsest fit)**.** *The coarsest granularity satisfying* (4) *minimises time subject to the memory constraint.*

Proof: Total I/O is invariant to granularity (Theorem 10), but the number of LOAD operations grows as the partition is refined. Each carries a fixed overhead λ (file access, launch), so the time is $|\mathcal{U}|\lambda + 2Ns/\mathrm{BW}$: the second term is constant and the first increases with $|\mathcal{U}|$. Hence the coarsest feasible partition is optimal. ∎

The consequence is that no user configuration is required. The system reads the architecture description, computes $\max_u|W_u|$ for each candidate granularity, and selects one; the same code path serves a 7B and a 70B model on the same card, differing only in the granularity chosen. When no supported granularity fits, the system reports this rather than failing opaquely—and, per Remark 11, it states explicitly when an unimplemented granularity would have sufficed.

XI. Experiments

A. Setup

All experiments run on a single desktop with an NVIDIA GTX 1050 (3.94 GB usable), 8 CPU cores, 15.5 GB of host memory, and an NVMe disk. The model is Qwen2.5-7B-Instruct [24] (7.62 B parameters, 28 layers, fp16), fine-tuned on Alpaca [25] with AdamW [26], $S = 256$, mixed precision [27], in PyTorch [28] with Transformers [20]. Two adapters are used to substantiate Corollary 5: LoRA [1] at ranks 1, 2 and 8, and S^3, a spectral–spatial–smooth adapter described in a companion paper. This work makes no claim

TABLE III
MEASURED GRADIENT DEVIATION FOR EACH EXACTNESS CLAIM. "TENSORS" IS THE NUMBER OF TRAINABLE GRADIENT TENSORS COMPARED.

Claim	Comparison	Tensors	$\max \lvert\Delta\rvert$
Theorem 3	streamed vs. resident	150	0.00
Theorem 13	layer- vs. batch-major (7B)	1400	0.00
Corollary 8	checkpoints on host vs. device	150	0.00
Theorem 10	sublayer vs. layer	150	0.00
Proposition 16	prefetch on vs. off	150	0.00

about the relative quality of the two; they appear here as evidence that USF is indifferent to the choice.

The implementation, the experiment configurations and the raw logs behind every number in this paper are available at a tagged, immutable revision [29]: development of the method continues elsewhere in that repository, but the tag pins the exact code that produced the measurements reported here. Each exactness claim in Table III corresponds to an executable test, and the figures are regenerated from the logged run rather than redrawn, so the reported deviations can be reproduced rather than taken on trust.

B. Exactness

Theorem 3, Theorem 13, Theorem 10 and Proposition 16 each assert that a transformation leaves gradients unchanged. Table III reports the measured deviation for each. Every entry is exactly zero.

The second row deserves emphasis. Theorem 3 can only be verified against a resident baseline on a model that *fits* resident, which on this hardware means a small one; we therefore verify it on a reduced configuration. Theorem 13, by contrast, compares two streaming schedules and is verifiable at full scale: on Qwen2.5-7B itself, over 1400 gradient tensors whose largest magnitude is 7.55×10^2, the deviation is exactly zero. The scale of the comparison matters because it excludes the possibility that agreement is an artifact of a small model.

C. Memory

Table IV reports peak device memory. Both adapters train the same frozen 7B model on the same card. The gap between them is precisely the $|\theta| \cdot s_{\text{opt}} + A$ term of (4)—S^3 keeps additional resident factors—and both satisfy the bound. That LoRA occupies *less* memory than S^3 is not a defect of the system but a confirmation of Corollary 5: USF imposes its bound irrespective of what is being adapted.

We report three measures rather than one because they differ substantially and only the largest is a fair test of the claim. The headline figure of 2.2 GB is the allocator peak; total device occupancy including the CUDA context reaches 2891 MB, or 71.7 % of the 4034 MB the card exposes to CUDA. The claim survives under the strictest measure, which is the reason to state it. Note also that peak memory is not monotone in adapter size—LoRA $r{=}1$ occupies more than $r{=}2$ under *resv* and *smi*—because allocator fragmentation, not parameter count, dominates these two measures at this scale; Theorem 7 bounds the allocation, not the allocator's caching policy.

TABLE IV
PEAK MEMORY AND TRAINABLE PARAMETERS (QWEN2.5-7B, 15.2 GB ON DISK). THREE MEMORY MEASURES ARE REPORTED: *alloc* IS THE PEAK OF THE PYTORCH ALLOCATOR, *resv* ADDITIONALLY COUNTS BLOCKS THE CACHING ALLOCATOR RETAINS, AND *smi* IS TOTAL DEVICE OCCUPANCY INCLUDING THE CUDA CONTEXT—THE STRICTEST MEASURE, AND THE ONE THAT MUST FIT. PERPLEXITY IS MEASURED ON 100 HELD-OUT ALPACA EXAMPLES, IDENTICAL DATA AND SEED ACROSS ROWS.

		Peak VRAM (MB)			Perplexity	
Adapter	**Trainable**	**alloc**	**resv**	**smi**	**before**	**after**
S^3	3.90 M	2269	2514	2891	9.52	4.46
LoRA $r{=}8$	20.19 M	1606	2072	2554	9.52	4.54
LoRA $r{=}2$	5.05 M	1598	1780	2161	9.52	4.57
LoRA $r{=}1$	2.52 M	1315	1938	2351	9.52	4.58
Card capacity (CUDA-visible)			4034			

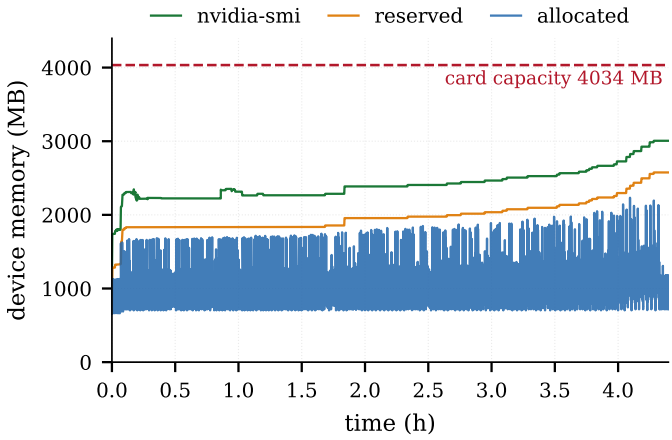

Fig. 2. Device memory over the full 4 h 16 min run of Qwen2.5-7B (114 766 samples at 100 ms). The sawtooth is Algorithm 1 itself: each tooth is one LOAD–compute–FREE cycle of a 0.43 GB decoder layer. The model is 15.2 GB on disk, yet nothing approaches the card's 4034 MB—the picture Theorem 7 predicts. The upward drift of the reserved curve is the caching allocator retaining freed blocks, not growth in what USF keeps resident.

D. Learning end to end

Table IV establishes that training reduces held-out perplexity for every adapter. A longer run—1100 Alpaca examples, one epoch, 275 micro-batches at $B{=}4$, hence 138 optimizer steps at $G{=}2$, 4 h 16 min—reduced held-out perplexity from 11.96 to 5.50 at a peak of 2233 MB, with no step skipped due to gradient overflow. Fig. 3 plots the trajectory. We report these as evidence that the system trains, not as a quality claim: by Theorem 3 the resulting model is the model that ordinary training would have produced, so there is nothing about quality that USF could change.

Fig. 2 shows the device-memory trace of that run, and is the most direct empirical picture of Theorem 7. The sawtooth is the mechanism itself: each tooth is one LOAD–compute–FREE cycle, and the envelope never approaches the 4034 MB capacity of the card despite the model being 15.2 GB on disk. The trace also exposes the allocator's behaviour: the reserved curve drifts upward as the caching allocator retains freed blocks, which is why we report all three measures in Table IV rather than the most flattering one.

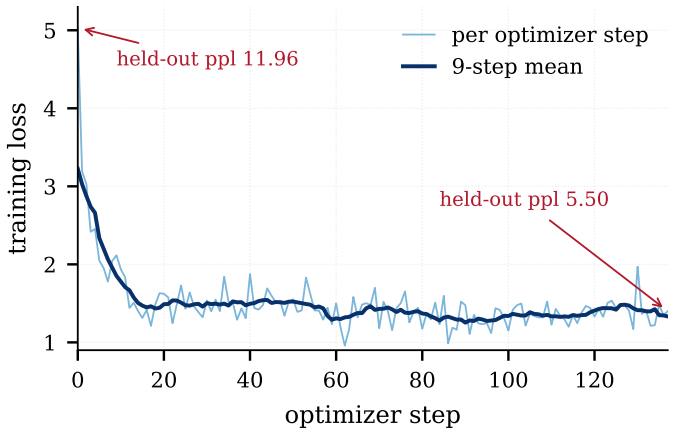

Fig. 3. Training loss over 138 optimizer steps (275 micro-batches at B=4, G=2) on 1100 Alpaca examples. Held-out perplexity, measured on 100 unseen examples before and after, fell from 11.96 to 5.50. No step was skipped for gradient overflow. By Theorem 3 this trajectory is the one ordinary residency training would have produced; we show it as evidence that the system trains, not as a quality claim.

TABLE V
BATCH-MAJOR VERSUS LAYER-MAJOR (QWEN2.5-7B, B=4, S=256, G=4, 16 EXAMPLES).

Schedule	Total	Per example	Peak VRAM
Batch-major	457.5 s	28.6 s	2524 MB
Layer-major	327.4 s	**20.5 s**	**2219 MB**
Gain		**1.40×**	−305 MB

E. Layer-major

Table V compares the two schedules on the 7B model over 16 examples with $G = 4$. Layer-major is 1.40× faster and uses *less* memory, the latter because its checkpoints are host-resident (Corollary 8). Proposition 14 predicts further gains at larger G; we did not measure them.

F. Comparison with quantization

The natural alternative for reducing resident memory is quantization. On this budget it is not merely inferior; it is infeasible. A 4-bit NF4 quantization of Qwen2.5-7B requires 3.25 GB for the transformer blocks, plus 1.09 GB for the embedding matrix and a further 1.09 GB for the language-modelling head, which this model does not tie to the embedding and which the reference implementation does not quantize: 5.43 GB against a card that exposes 3.94 GB. The prediction is borne out—the load terminates in an out-of-memory error—while USF trains the same model at an allocator peak of 2.27 GB.

The comparison is deliberately drawn in quantization's favour. The 5.43 GB counts weights alone, before optimizer state, activations or the CUDA context, and is therefore a lower bound on what 4-bit would need; the USF figures are full measured runtime. The gap is wide enough that the asymmetry does not matter, which is why we can state the conclusion as a separation of feasibility rather than a preference. The two techniques are composable and address different axes: quantization purchases memory with accuracy, USF purchases it with time, and only the latter is exact (Corollary 4).

TABLE VI
SCOPE OF THE CLAIMS. THE THREE ROWS HAVE DIFFERENT EPISTEMIC STATUS.

Model	Status	Basis
Qwen2.5-7B	**Demonstrated**	Executed: 2.2 GB peak, ppl $9.52 \to 4.46$, $\max \lvert\Delta\rvert = 0$
Llama-3-70B	**Predicted by the bound; not executed**	1.59 GB per layer $<$ 3.94 GB at an *implemented* granularity (Table I); the weights were not available to us
Llama-3-405B	**Theoretical; requires an unimplemented extension**	Needs matrix granularity (1.62 GB); see Remark 11

XII. SCALING ANALYSIS

Table VI states what this paper establishes at each scale, and on what basis. We consider the distinction important enough to tabulate rather than to leave to prose.

Using (10)–(11) with the constants calibrated in Section IX, and layer-major with $G = 8$, the projected cost of 1000 examples is 4.0 h for 7B, 41.7 h for 70B, and 245 h for 405B; all three are compute-bound, so the projections are governed by Φ rather than by disk bandwidth. The honest reading of these numbers is that USF removes the memory barrier and substitutes a time barrier. A 70B model becomes feasible as a demonstration on a consumer card; it does not become a practical workflow. The practical regime for this class of hardware is 7B–13B, and we say so in Section XIII rather than leaving it to be discovered.

XIII. LIMITATIONS

Time replaces memory. USF is substantially slower than resident training; the whole method is a trade, and Section XII quantifies it. We claim feasibility on hardware where the alternative is nothing at all, not competitiveness with a data-centre GPU.

Local storage is required. The method presumes the weights are on a local disk, and BW bounds the I/O term of (10).

The PEFT hypothesis is load-bearing. Theorem 7 treats $|\theta| \cdot s_{\text{opt}}$ as small. With many trainable parameters the optimizer state ceases to be negligible and partitioning it becomes the dominant concern—the regime of ZeRO [10], not of this paper. USF is a method for PEFT, not for pre-training.

Matrix granularity is not implemented (Remark 11), which is why Table VI assigns 405B a weaker status than 70B.

Constants are hardware-specific. The cost *model* of Section IX is general; the values of Φ and BW used in Tables II and VI are not, and would need recalibration elsewhere.

Inference is unaffected. USF is a training method.

XIV. Conclusion

We have shown that the memory required to fine-tune a frozen language model is not a function of the model's size. It is a function of the model's largest streamable unit, because reverse-mode differentiation is local and residency is therefore a matter of scheduling rather than of correctness. The resulting method is exact—verified at zero deviation on the 7B model itself— adapter-agnostic, and governed by a cost model accurate to within 6%. Its consequence is a displacement rather than an elimination of cost: model size becomes a question of time and storage. On a consumer GPU that a data-centre would consider obsolete, a 7B model can now be fine-tuned exactly; the same bound, at an implemented granularity, places a 70B model within reach of the same card. What remains scarce is time, and time is a more tractable adversary than a memory wall.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[2] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning (ICML)*, 2019.

[3] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[6] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[7] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. SIAM, 2008.

[8] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018.

[9] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen, "DoRA: Weight-decomposed low-rank adaptation," in *International Conference on Machine Learning (ICML)*, 2024.

[10] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.

[11] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, and Y. He, "ZeRO-Offload: Democratizing billion-scale model training," in *USENIX Annual Technical Conference (ATC)*, 2021.

[12] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan, "Accelerate: Training and inference at scale made simple, efficient and adaptable," https://github.com/huggingface/accelerate, 2022.

[13] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," *arXiv preprint arXiv:1604.06174*, 2016.

[14] A. Griewank and A. Walther, "Algorithm 799: Revolve: An implementation of checkpointing for the reverse or adjoint mode of computational differentiation," *ACM Transactions on Mathematical Software*, vol. 26, no. 1, pp. 19–45, 2000.

[15] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[16] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[17] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[18] Z. Jia, M. Zaharia, and A. Aiken, "Beyond data and model parallelism for deep neural networks," in *Proceedings of Machine Learning and Systems (MLSys)*, 2019.

[19] A. Grattafiori, A. Dubey, A. Jauhri *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[20] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue *et al.*, "Transformers: State-of-the-art natural language processing," in *Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020.

[21] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, 2024.

[22] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.

[23] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[24] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2025.

[25] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford Alpaca: An instruction-following LLaMA model," https://github.com/tatsu-lab/stanford_alpaca, 2023.

[26] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[27] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia *et al.*, "Mixed precision training," in *International Conference on Learning Representations (ICLR)*, 2018.

[28] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[29] "USF: Reference implementation of unit-streamed fine-tuning," https://github.com/Anndy-bit/lowrank-field-adapters/tree/v1.0-usf-paper1, 2026, source code, experiment configurations and run logs. Tagged release `v1.0-usf-paper1` (commit `9a86aed`): the exact revision that produced every measurement reported here.